Postponing the Glitches is Not Enough

A Critical Analysis of the DATE 2024 E-ISW Masking Scheme

Amir Moradi

Technische Universität Darmstadt, Darmstadt, Germany
{firstname.lastname}@tu-darmstadt.de

Abstract.
The Enhanced ISW (E-ISW) masking scheme, recently proposed at DATE 2024, was introduced as a refinement to the classical ISW construction to restore provable security guarantees in hardware implementations affected by glitches. By enforcing input-complete gate evaluations through the use of artificial delays, E-ISW seeks to mitigate the glitch-induced leakage that compromises standard masking techniques. However, in this work, we demonstrate that this modification is fundamentally insufficient to ensure robust side-channel resistance in realistic hardware environments. We conduct a detailed analysis and present concrete examples where E-ISW fails to prevent information leakage, even when the prescribed countermeasures are correctly applied. These vulnerabilities arise due to deeper conceptual shortcomings in the design, particularly the absence of compositional reasoning about the interaction between glitches and masking. Our results show that the security claims of E-ISW do not hold in practice, and they expose critical limitations in relying on heuristic delay-based fixes without formal and compositional proofs of security. This study serves as a cautionary note for the cryptographic engineering community, emphasizing the necessity of rigorous validation when proposing enhancements to established secure computation techniques.

Keywords: Side Channel Analysis, Masking, ISW, Glitches

1 Introduction and Background

More than two decades after their introduction [KJJ99], side-channel analysis attacks remain one of the most potent threats to the security of cryptographic implementations. These attacks exploit unintended physical leakages – such as power consumption, electromagnetic radiation, or timing variations – emitted during computation, enabling adversaries to extract secret information without breaking the underlying cryptographic algorithm. In response, a wide range of countermeasures have been proposed over the years, many of which are ad hoc or heuristic in nature. Most of these fall into the category of *hiding* techniques, which aim to reduce the signal-to-noise ratio of the observable leakage by making it less correlated with the manipulated data [MOP07]. While often effective in practice, such techniques typically lack rigorous, formal guarantees of security.

To address this gap, the cryptographic community has turned its attention to *masking schemes*, which offer a stronger theoretical foundation. Masking seeks to randomize intermediate values of a computation in such a way that the adversary, even when able to observe certain points within a circuit, gains no useful information about the secret [CJRR99]. The security of these schemes can, in principle, be formally proven – provided that the adversary model is well-defined. Without a clear model specifying the adversary's capabilities, no meaningful notion of security can be claimed.

The earliest and arguably most intuitive model is the *noisy leakage model*, in which the adversary observes leakage that is probabilistically related to intermediate values via some noisy function – commonly, a noisy Hamming weight or Hamming distance model. This approach has been widely adopted in empirical studies evaluating the effectiveness of side-channel countermeasures [PM09, HRG14]. However, the amount and quality of noise in real devices depend on many factors, including the physical platform, measurement setup, and the adversary's signal processing expertise. Thus, while this model is intuitive and aligns closely with practical attack scenarios, it is difficult to use for formal proofs, as the noise level is not under precise control.

To enable rigorous reasoning, more abstract models have been developed to formalize an adversary's observational power. The most prominent among these is the *probing security model* introduced by Ishai, Sahai, and Wagner [ISW03]. In this model, the adversary is allowed to place up to t probes on internal wires of a cryptographic circuit, thereby observing the values of t intermediate signals during execution. A scheme is said to be t-probing secure if any such observation reveals no information about the secret, in an information-theoretic sense.

Crucially, it has been shown that probing security implies noisy leakage security under reasonable assumptions. Specifically, an implementation that is secure against a t-probing adversary is also secure against an adversary limited to analyzing up to the t-th statistical moment of the leakage distribution [BDF+17]. For example, an implementation that achieves 1-probing security should, in principle, resist all first-order side-channel analysis attacks, such as Differential Power Analysis (DPA) or Correlation Power Analysis (CPA) [KJJ99, BCO04], which rely on comparing the mean of the leakage.

While this correspondence makes the probing model a powerful tool for designing implementations with provable security, it comes at a cost: schemes that satisfy probing security often incur significant performance and area overheads. Interestingly, the implication from probing to noisy leakage security is one-directional. That is, an implementation that empirically resists all first-order side-channel analysis attacks is not necessarily secure in the 1-probing model. This asymmetry underscores the importance of formal modeling and highlights potential pitfalls when relying solely on empirical validation for side-channel resistance.

To enable secure computation under the probing model, Ishai, Sahai, and Wagner proposed a construction for implementing the basic logical operations XOR and AND in a way that it remains secure against any adversary capable of placing up to t probes on the circuit [ISW03]. These constructions, now commonly referred to as *ISW gadgets*, provide a composable method for building arbitrary circuits with t-probing security, since any Boolean function can be realized using only XOR and AND gates. The security of ISW gadgets is grounded in rigorous theoretical proofs, and under the probing model, their guarantees are sound.

However, an underlying key assumption of the ISW security proof is the ideal nature of logical operations: each gate is assumed to behave atomically, meaning its inputs and outputs are accessed or probed only when they are stable, not while they are in transition or being evaluated. This assumption is relatively plausible in software settings, where instructions – including logical operations – are executed sequentially, and intermediate values can be treated as stable between operations.

Yet, even in software, this assumption does not always hold in practice. Modern microprocessors exhibit complex micro-architectural behavior that can invalidate the aforementioned ideal assumption. For instance, the process of fetching operands from the register file, feeding them into the Arithmetic Logic Unit (ALU), and writing the results back may involve multiple intermediate steps, each of which could produce observable side-channel leakage. Additionally, micro-architectural features such as pipelining, shadow registers, and register renaming further complicate the timing and visibility of internal

values. The situation becomes even more precarious in the presence of speculative execution and branch prediction, where instructions may be partially or fully executed out-of-order, depending on runtime conditions.

Several works have demonstrated that probing-secure pseudo-code implementations can still leak sensitive information when compiled and run on real-world microprocessors [BWG+22]. These empirical results highlight the disconnect between theoretical models and hardware realities. Importantly, they do not invalidate the ISW scheme itself, but rather show that implementing ISW securely on practical platforms requires additional considerations beyond those captured by the probing model.

This discrepancy stems from the fact that the t-probing model does not account for micro-architectural side effects. Moreover, accurately modeling such effects is infeasible in most cases, as the internal architecture and execution details of commercial microprocessors are proprietary and undocumented. Recent studies have explored these challenges and reported leakage in practice despite the use of theoretically secure masking schemes [ZM24, GD23, HHB24, MPW22]. Together, these findings underscore the importance of bridging the gap between theoretical security models and practical implementations – particularly when deploying countermeasures on complex and non-transparent hardware platforms.

In hardware, the situation differs significantly from software, and addressing side-channel leakage presents unique challenges. Over the past two decades, researchers have devoted substantial effort to adapting security models and developing specialized hardware gadgets to enable masked implementations that resist physical attacks. Early studies revealed a sobering reality: masked hardware circuits, when implemented naively, can leak as much as – or even more than – their unprotected counterparts [MPO05]. The root cause of this phenomenon was identified as *glitches* which are unintended signal transitions caused by differences in signal propagation delays within combinational logic.

To tackle this problem, alternative design principles and circuit styles have been proposed. Notable among them are Threshold Implementations (TI) [NRS11] and Domain-Oriented Masking (DOM) [GMK16], both of which aim to build hardware circuits that remain secure even in the presence of glitches. These approaches avoid leakage by enforcing strict input-completeness and disallowing early evaluation, ensuring that all inputs to a gate are known and stable before any computation occurs.

To formalize and reason about these new designs, the *robust probing model* was introduced [FGP+18]. This model extends the classical probing framework by recognizing that in hardware, a probe on a signal can indirectly reveal information from other parts of the circuit due to physical effects like glitches, signal transitions, and capacitive coupling. In this model, a single probe can *expand* into a set of related probes, making it a more realistic abstraction of actual leakage.

Specifically, the robust probing model defines three propagation rules: glitches, transitions, and coupling. Glitch propagation captures the idea that if a probe is placed on the output of a combinational gate, it must also be considered to implicitly probe all the gate's inputs – whether they come from primary inputs or from preceding registers. This reflects the fact that intermediate signal instability during computation can leak sensitive information. Transition-based propagation accounts for the behavior of registers: since side-channel leakages depend not only on the new value being stored but also on the value being overwritten on a register, a probe on a register output must also reach its input. Lastly, coupling refers to the electrical interaction between adjacent wires; a probe may reveal information from neighboring signals due to capacitive or inductive effects. While this last aspect is less studied, it presents a significant challenge, as modeling such coupling requires knowledge of the physical layout and placement of wires in a fabricated chip – details that are typically unknown or extremely difficult to model precisely.

With the robust probing model in place, it has become feasible to design masked hardware circuits that are not only secure in theory but also in real-world settings. This

has led to the development of systematic approaches for constructing hardware circuits that resist side-channel leakage under realistic adversary models [SM21, CGLS21, KM22, CSV24]. In retrospect, earlier countermeasures such as TI and DOM can now be reinterpreted through the lens of the robust probing model. It is possible to formally prove their security when the model is extended to include glitch and transition propagation.

Furthermore, in-depth studies – such as [RBN+15] – have demonstrated how ISW gadgets can, in principle, be implemented securely in hardware by addressing glitches and transitions explicitly. A common design principle underlying all these approaches is the strategic insertion of registers inside or around gadgets. This design choice is critical to cut off the propagation paths of glitch-extended probes, thereby preserving the assumptions required for robust probing security.

However, this security-oriented design comes at a cost. The inclusion of extra registers increases the latency of computation, often significantly so. As a result, recent research has turned toward *low-latency* hardware masking, aiming to reconcile robust security guarantees with performance efficiency [GIB18, KSM22, VDB+24, ZSS+21]. Other approaches explore fundamentally different design styles. For instance, dual-rail pre-charge logic has been employed to construct low-latency and more balanced masked circuits [SBHM20, NGPM22]. These logic families aim to reduce the circuit's susceptibility to glitches by enforcing controlled timing and balanced switching activity.

Our Contributions

Among the most recent contributions in this space is the *Enhanced ISW (E-ISW)* scheme, proposed at DATE 2024 [TBE+24]. E-ISW aims to *repair* the original ISW gadget for hardware implementations by introducing *ad hoc* delay-balancing techniques. The main idea is to manually synchronize gate evaluations by inserting carefully tuned delay elements. This is intended to enforce input-completeness and prevent early evaluation, thereby mitigating the leakage caused by glitches.

While this method appears attractive – especially given its promising simulation results in automated leakage detection tools – our findings in this paper show that such an approach is fundamentally flawed. Despite the intuitive motivation and superficial leakage suppression, E-ISW fails to provide rigorous or compositional guarantees. As we demonstrate through concrete examples, E-ISW can still leak sensitive information even when its core conditions are satisfied. Thus, relying solely on artificial delays to mitigate glitch-induced leakages is insufficient to achieve meaningful security in practice.

In summary, this work presents a thorough analysis of the E-ISW construction and uncovers fundamental weaknesses in its design rationale. We show that the core principle behind E-ISW – manually controlling gate evaluation timing via artificial delays – is insufficient to suppress all exploitable glitch-induced leakages. In particular, we demonstrate that even when E-ISW's delay-balancing constraints are correctly enforced, the resulting circuits may still exhibit *first-order leakage* under realistic adversarial models. This undermines the security claims of the scheme and enables practical key recovery attacks.

Our analysis is not purely theoretical. Through FPGA-based experiments, we construct concrete case studies in which E-ISW fails to uphold the essential invariants of secure masked computation – specifically, the independence between the leakage and the underlying secret. These violations lead to observable and exploitable side-channel leakages despite the application of the proposed countermeasure.

Beyond the specific weaknesses of E-ISW, our findings illustrate a broader and more critical point: hardware-level fixes that are heuristically motivated – even when grounded in a high-level understanding of masking – cannot replace *compositional reasoning* and *formally verified designs*. Without rigorous guarantees, such countermeasures may inadvertently introduce new vulnerabilities while offering only an illusion of security. E-ISW, although appealing from an engineering standpoint, ultimately fails to provide the level of

protection it promises.

This work serves as a cautionary tale for the design of future masking schemes in hardware. It emphasizes the need for rigorous security proofs and highlights the limitations of relying solely on simulation-based leakage detection, which – even when positive – cannot be taken as conclusive evidence of security.

Organization

The remainder of this paper is organized as follows. In Section 2, we review the ISW construction, its limitations in hardware due to glitches, and the development of DOM as a robust alternative. We then describe the E-ISW proposal and its intended mitigation strategy. Section 3 presents our analysis of E-ISW, both from a theoretical perspective under various conditions and through practical case studies implemented on FPGA to evaluate its leakage behavior. Finally, Section 4 concludes the paper with a summary of our findings and their implications for future masking designs.

2 ISW, Problem, and the DATE 2024 Solution

Throughout this paper, we adopt a consistent notation to facilitate readability and clarity. Single-bit random variables are denoted by lowercase letters such as x, while vectors of such variables are written in uppercase, e.g., X. The individual components of a vector are accessed using subscripts, as in $X = \langle x_{n-1}, \ldots, x_0 \rangle$. When discussing masking schemes, we use superscripts to indicate shares – e.g., x^0 refers to the first share of a masked variable x, and X^1 to the second share of the masked vector X.

In the remainder of this section, we begin by briefly revisiting the ISW masking scheme, focusing on its construction and its limitations under the robust probing model. We then recall the DOM technique as a response to these limitations. Finally, we describe the E-ISW approach, which was proposed to restore the security of ISW gadgets in hardware through timing control while avoiding the registers forced by DOM. To keep the discussion focused and aligned with the scope of E-ISW, we limit our analysis to the 2-input AND operation, which forms the core building block of both ISW and its enhancement. However, our analyses, claims, and conclusions are general and apply to arbitrary functions and at higher security orders constructed using these techniques.

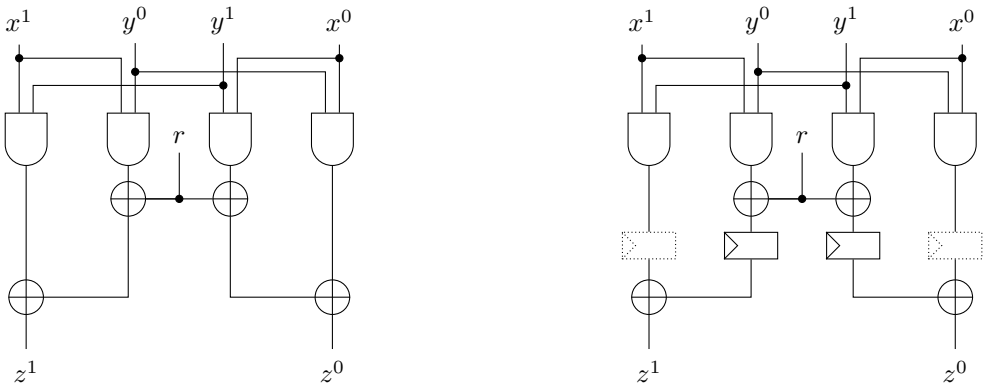
2.1 ISW

Similar to many other masking schemes, ISW operates on the basis of Boolean masking. More precisely, each secret random variable[1] is split into Boolean shares $(x^0, \ldots, x^d)$ such that their bitwise XOR reconstructs the original value, i.e., $x = \bigoplus_{\forall i} x^i$, where d denotes the masking order. For every cipher execution, each primary input x – such as the plaintext or key in an encryption algorithm – is split into $d+1$ shares. This is achieved by randomly sampling d of the shares (e.g., $x^1, \ldots, x^d$) independently and uniformly, and then computing the final share $x^0 = x \oplus \bigoplus_{i>0} x^i$ to preserve correctness.

The cryptographic function then operates entirely on the masked inputs, ultimately producing a masked version of the output (e.g., ciphertext), which can be unmasked to reveal the final result. For simplicity and without loss of generality, we restrict our discussion in this section to first-order masking (i.e., $d = 1$), meaning each sensitive variable is split into two shares.

Boolean masking enables XOR operations to be implemented in a straightforward and share-wise manner. Suppose we want to compute $z = x \oplus y$, where x and y are masked as

[1] Here, *secret* refers not only to cryptographic keys but also to all intermediate values computed during the execution of a cipher, as they must remain confidential.



(a) ISW [ISW03] (b) DOM [GMK16]

Figure 1: ISW and DOM AND gadgets.

(x^0, x^1) and (y^0, y^1), respectively. Thanks to the linearity of XOR under Boolean masking, each share of z can be directly computed as $z^0 = x^0 \oplus y^0$ and $z^1 = x^1 \oplus y^1$.

The situation is more subtle for the AND operation $w = x\,y$. When expanded in terms of the shares, we have

$$w = (x^0 \oplus x^1)(y^0 \oplus y^1) = x^0y^0 \oplus x^0y^1 \oplus x^1y^0 \oplus x^1y^1.$$

This expression must be split into two shares (w^0, w^1) such that $w = w^0 \oplus w^1$. A naive assignment like $w^0 = x^0y^0 \oplus x^0y^1$ and $w^1 = x^1y^1 \oplus x^1y^0$ would maintain correctness, but not security. For instance, $w^0 = x^0(y^0 \oplus y^1) = x^0\,y$ implies that if an adversary probes w^0 and observes a '1', they can directly infer $y = 1$, which violates the masking security.

To prevent such leakage, the ISW scheme introduces an additional random variable r, referred to as fresh randomness or online randomness, sampled uniformly and independently for each AND operation. With this, the secure sharing becomes

$$w^0 = x^0y^0 \oplus (x^0y^1 \oplus r), \quad w^1 = x^1y^1 \oplus (x^1y^0 \oplus r)$$

Here, the order of operations is critical: the terms x^0y^1 and x^1y^0 must first be blinded with r before being XORed with the remaining terms. This structure ensures that each share individually leaks no information about x or y under the probing model.

However, when such a construction is implemented in hardware (see Figure 1(a)), the temporal behavior of gate evaluations introduces glitches. A probe placed on w^0 may observe transient switching activity originating from both XOR and AND gates. These glitches violate the expected computation order and may leak sensitive intermediate values – thereby undermining the intended masking security.

One of the well-known and widely adopted techniques to address the shortcomings of ISW in hardware is DOM, introduced in [GMK16]. The core idea of DOM is to isolate potentially leaky computations by enforcing temporal separation between critical operations, thereby mitigating the impact of glitches that may arise when multiple gate transitions overlap.

More concretely, DOM targets the vulnerability in ISW stemming from the early combination of terms like x^0y^0 and $x^0y^1 \oplus r$. If these are computed in close temporal proximity or within the same combinational path, glitches may reveal sensitive correlations (e.g., partial knowledge of y), thereby violating first-order security. To counteract this, DOM introduces register stages directly after computing the blinded terms (e.g., $x^0y^1 \oplus r$), as illustrated in Figure 1(b). These registers prevent glitch propagation by ensuring that values entering the final XOR gates are stable and temporally separated.

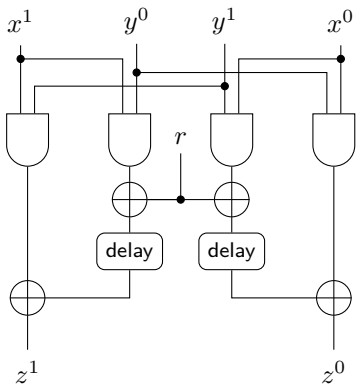


Figure 2: E-ISW AND gadget [TBE+24].

Note that some of the registers shown in dashed lines in Figure 1(b) are not strictly necessary for security. They are used to enable pipelined implementations that process independent input tuples on successive clock cycles. In serialized implementations where inputs are handled sequentially, such registers can be omitted to save area and power.

Assuming independent sharing of x and y and proper use of fresh randomness, DOM satisfies the glitch-extended robust probing model [FGP+18]. In this model, probes conservatively propagate through combinational logic to account for glitches. DOM thus provides a reliable hardware-level building block for secure masked circuits.

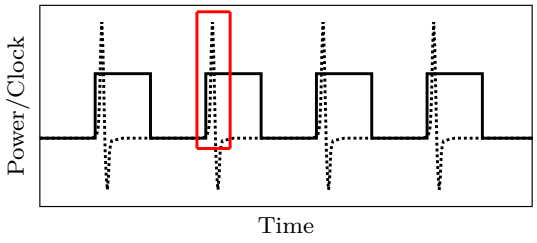
The main downside of DOM is increased latency and area. For example, applying DOM to the AES S-box results in a minimum of four clock cycles for masked evaluation [GMK16], due to the required sequencing and intermediate register stages. Despite this, DOM remains one of the most robust and practical approaches for secure masking in glitch-prone hardware platforms.

2.2 E-ISW

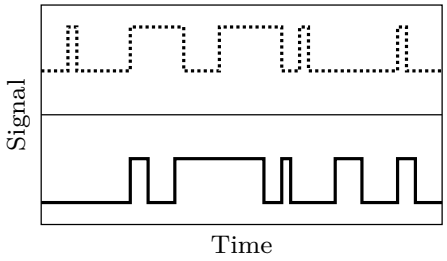
With the goal of avoiding the added latency introduced by countermeasures like DOM and preventing the aforementioned issues of ISW when implemented in hardware, the authors of [TBE+24] proposed a modification to the ISW scheme called Enhanced ISW (E-ISW). The motivation behind E-ISW is to preserve the theoretical benefits of ISW, while mitigating its practical vulnerabilities in hardware, especially those arising from glitch-induced leakage due to incorrect evaluation order.

To this end, E-ISW inserts delay elements into the combinational logic path of the circuit (see Figure 2), with the specific aim of controlling the timing at which certain gate inputs arrive. The underlying idea is that by artificially delaying the arrival of the second operand in the final XOR gate – for instance, computing $x^0y^0 \oplus \mathsf{delayed}(x^0y^1 \oplus r)$ – one can enforce a more predictable order of signal evaluation. The authors argue that such timing control helps suppress the harmful glitches that might otherwise arise from near-simultaneous switching activity, and thus reduces the risk of unintended information leakage about y.

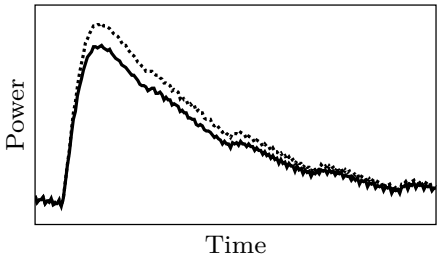
E-ISW is appealing from an engineering perspective, as it attempts to reduce vulnerability without sacrificing throughput or requiring additional clock cycles. However, as we will demonstrate in this work, this intuition does not hold in practice under realistic leakage models. In particular, the mere insertion of delay elements is insufficient to prevent exploitable glitch propagation, and may result in a false sense of security in the absence of rigorous analysis or compositional proofs.



(a) Clock signal (solid line) and the corresponding dynamic power consumption trace (dashed line).



(b) Two exemplary glitch patterns occurring within one clock cycle.



(c) Power consumption peaks associated with the glitch patterns of Figure 3(b).

Figure 3: Dynamic power consumption of a CMOS circuit.

3 Analysis

In this section, we present a comprehensive analysis of the E-ISW scheme, beginning with a theoretical investigation under various conditions and continuing with a practical evaluation based on FPGA implementations. Our theoretical analysis highlights fundamental flaws in the assumptions underpinning E-ISW, particularly regarding its ability to prevent leakage in the presence of glitches. These issues are then substantiated by experimental results, where we demonstrate concrete cases in which E-ISW fails to preserve the security guarantees expected from a first-order masking scheme. Together, these analyses provide strong evidence that E-ISW does not achieve robust protection under realistic adversarial models.

3.1 Theoretical

Before proceeding with the analysis of the E-ISW scheme, we first elaborate on the role of glitches in SCA and how these effects are captured by the robust probing model. Glitches originate from asynchronous input changes at a logic gate – when the inputs of a gate switch at different time instances due to varying path delays. Since a combinational circuit is composed of interconnected gates, any input change will propagate through the circuit with varying latencies, causing transient logic hazards (i.e., glitches) on internal nodes.

From the viewpoint of SCA, these glitches have direct implications. The dynamic power consumption of CMOS circuits, which is the primary source of leakage exploited in power-based SCA attacks, is mainly induced by output transitions of gates. As each glitch corresponds to a logic transition, it contributes to the total power consumption. Importantly, the pattern of glitches depends not only on the current input values but also on the timing of their arrival, thereby encoding sensitive information in the temporal dimension.

Power measurements in SCA are typically obtained by monitoring the voltage drop across a shunt resistor inserted into the $\mathtt{V_{dd}}$ or `GND` path, using high-resolution oscillo-

scopes [MOP07]. The overall measurement setup – including the integrated circuit (IC) packaging, printed circuit board (PCB), shunt resistor, and oscilloscope probes – acts as a low-pass filter. As a result, all glitches occurring within a clock cycle contribute collectively to the observable power trace. This phenomenon is illustrated in Figure 3. A similar behavior applies to Electro-Magnetic (EM) radiation-based measurements, although the higher bandwidth of EM probes allows for capturing finer temporal effects.

Therefore, in the glitch-extended robust probing model, a probe placed on the output of a combinational circuit must be assumed to implicitly observe all of its inputs, since their relative timing and values collectively determine the output transitions and hence the observable leakage.[2] Although this model is considered conservative,[3] it provides a tractable abstraction that has been widely adopted in the literature. More relaxed and precise models have been also proposed, such as the timing-aware variant introduced in [MM24], but the classic glitch-extended model remains prevalent due to its simplicity and soundness in worst-case reasoning.

In the following, we present a systematic analysis of the ISW and E-ISW masking schemes with respect to their vulnerability to glitches. Our aim is to evaluate whether E-ISW effectively mitigates data-dependent transient behavior – particularly in comparison to its predecessor, ISW, and to more principled countermeasures like Domain-Oriented Masking (DOM). The analysis is carried out under a simplified, yet insightful, modeling framework designed to isolate structural weaknesses that can give rise to exploitable leakage in masked hardware circuits.

To focus solely on the effects of signal arrival times and glitch propagation, we adopt the following simplifying assumptions:

- Each logic gate (XOR or AND) is associated with a fixed, gate-specific propagation delay. This allows us to trace relative timing between signals without the need for a full timing characterization.
- All primary inputs to the gadget are initialized to zero. This setup eliminates switching activity at the inputs, enabling us to focus exclusively on the internal behavior and glitch dynamics triggered by controlled transitions.
- We examine selected input-to-output transitions and trace their propagation through the circuit, categorizing the resulting intermediate signal transitions and the nature of the glitches they induce.

These assumptions are intentionally chosen to simplify the reasoning while preserving the essence of the glitch behavior relevant to the robust probing model. As such, this analysis should be viewed as identifying worst-case or structurally inherent flaws rather than capturing every possible implementation detail.

3.1.1 Analysis Under Delayed Refresh

We begin by revisiting the original ISW gadget under the assumption – also adopted in for E-ISW – that the fresh randomness input r arrives later than the primary data inputs x^0, x^1, y^0, and y^1. This setup models the realistic scenario in hardware where signal arrival times may vary due to routing or gate delays. In this case, the partial product terms x^0y^0 and x^0y^1 are computed before the arrival of r, meaning that the blinding effect of the fresh randomness is delayed. Consequently, a probe placed on the intermediate signal $x^0y^0 \oplus x^0y^1$ may observe sensitive data before any effective masking takes place.

[2] Any synchronous digital circuit can be modeled as a Mealy machine, where a single combinational circuit is driven by the current state (i.e., register outputs) and primary inputs. Hence, it can be assumed that the probes are placed at the input of registers, i.e., the output of the combinational circuit.

[3] For instance, if one input of an AND gate remains constant at 0 for consecutive cycles, probing its output does not yield information about the other input.

To illustrate this leakage more concretely, we refer to Table 1, which reports the behavior of various intermediate and output signals for different combinations of input values considering the circuits shown in Figure 1(a) and Figure 2. In this table, the upper half of the rows corresponds to the case $y = 0$, while the lower half represents $y = 1$. For the masking scheme to be secure, the set of signal patterns resulting from $y = 0$ must be statistically indistinguishable from those resulting from $y = 1$. That is, the two sets (corresponding to $y = 0$ and $y = 1$) should be equal up to permutation, ensuring that no probe – placed on either intermediate or output wires – can gain information about the unmasked value of y.

As a concrete example, consider the behavior of the term x^0y^1 in Table 1. Because this term is independent of the unmasked value y, its glitch patterns appear uniformly across both halves of the table. This property demonstrates what statistical invariance looks like in a secure masking setting. However, this invariance does not hold for the output signal z^0 of the ISW construction. Due to the not-yet-blinded interaction between x^0y^0 and x^0y^1 prior to the arrival of r, the resulting glitch patterns in z^0 differ depending on the value of y. This discrepancy reveals a clear data-dependent leakage path, confirming the known vulnerability of ISW in the presence of glitches – a conclusion consistent with prior work.

Next, we turn our attention to E-ISW under the same delayed-refresh scenario. The central idea behind E-ISW is to delay the term $x^0y^1 \oplus r$ in hardware, aiming to enforce a specific ordering of operations that preserves the masking invariant. This is reflected in the second-to-last column of Table 1, where we observe that – assuming the delay is sufficiently long – the signal $x^0y^1 \oplus r$ exhibits glitch patterns that are independent of the unmasked input y. This suggests that the delayed blinding step is effective in isolation.

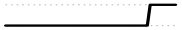
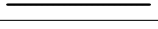
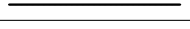
However, the final output signal z^0 of the E-ISW circuit, shown in the rightmost column, tells a different story. Despite the insertion of the delay element, z^0 continues to exhibit data-dependent glitch patterns that differ between the $y = 0$ and $y = 1$ cases. This indicates that, while the delayed term itself may behave securely, its interaction with the rest of the circuit still results in exploitable leakage. In other words, E-ISW does not eliminate the root cause of data-dependent glitches – the violation of separation between masked computations. Although its use of delay elements appears to mitigate part of the problem, it ultimately fails to restore the masking security that ISW loses in hardware. This result highlights a broader concern: engineering-level timing tricks, while intuitively appealing, do not guarantee security unless grounded in a compositional and model-driven framework.

3.1.2 Analysis Under Early Refresh

To assess whether the timing of the refresh signal r influences the effectiveness or vulnerability of E-ISW, we repeat our analysis under a new assumption: this time, r arrives *before* the primary data inputs x^0, x^1, y^0, and y^1. This scenario reflects an optimistic hardware setup where fresh randomness is available early enough to blind sensitive intermediates as soon as they are generated. Intuitively, one might expect this timing to preempt glitch-induced leakage by ensuring that the random mask r is already in place before any partial products are evaluated.

However, as illustrated in Table 2, this early arrival of r has no meaningful effect on the output glitch behavior. Despite the change in relative timing, both ISW and E-ISW continue to exhibit data-dependent transitions at the output z^0. The glitch patterns for $y = 0$ remain distinguishable from those for $y = 1$, demonstrating that sensitive information still leaks through transient gate behavior.

Table 1: Glitch patterns of ISW and E-ISW AND gadgets, assuming r arrives after all other primary inputs.

y	y^0	y^1	x^0	r	$x^0\,y^0$	$x^0\,y^1$	$x^0\,y^1 \oplus r$	ISW z^0	delayed $(x^0\,y^1 \oplus r)$	E-ISW z^0
0	0	0	0	0						
				1						
			1	0						
				1						
	1	1	0	0						
				1						
			1	0						
				1						
1	0	1	0	0						
				1						
			1	0						
				1						
	1	0	0	0						
				1						
			1	0						
				1						

This reinforces a central insight: the leakage is not solely a function of when the randomness arrives, but rather a consequence of how the inputs and the intermediates interact structurally within the circuit. In both ISW and E-ISW, the output z^0 is ultimately computed as the XOR of multiple terms involving both data and randomness. Glitch propagation across these terms can still result in data-dependent signal activity, even when r is present early.

Therefore, we conclude that E-ISW's security does not improve simply by adjusting the delay profile. Its masking strategy remains vulnerable to glitch-induced leakage, and the proposed delay-based mitigation in [TBE+24] does not achieve robustness under the glitch-extended robust probing model. This further supports the need for more principled, provably secure design approaches that explicitly account for signal timing and compositional behavior in hardware.

3.1.3 Comparison with DOM

For completeness, we analyze the behavior of the DOM scheme [GMK16], which was explicitly designed to achieve provable security under the glitch-extended robust probing model, albeit at the cost of increased latency. Using the same delay-based glitch model as in our previous analyses, we examine whether DOM exhibits any data-dependent glitch behavior that could undermine its masking guarantees.

The results of this analysis are presented in Table 3. Unlike ISW and E-ISW, DOM inserts dedicated registers at strategic points within the computation – particularly after the application of fresh randomness – to enforce strict separation of signal domains and eliminate harmful signal interactions (see Figure 1(b)). This ensures that the effect of randomness is fully established before any subsequent computation occurs.

As seen in the table, the glitch patterns observed for all internal and output signals are invariant with respect to the unmasked input y. That is, the set of transitions observed when $y = 0$ is identical (up to permutation) to those when $y = 1$. This holds regardless of the relative arrival time of r, confirming that DOM's masking construction effectively neutralizes the impact of signal timing on leakage.

It is worth emphasizing that the registers introduced in DOM serve a dual purpose. While their primary role is to block glitch propagation by separating combinational stages, they also implicitly enforce synchronization of sensitive operations. Since all registers are assumed to be triggered by a common clock edge, the transitions they initiate are decoupled from the relative arrival times of data inputs. This synchronization guarantees that the computation of masked outputs proceeds in a timing-invariant manner, independent of the specific data values or gate delays. Thus, the use of synchronous registers not only mitigates glitches but also aligns the timing of operations in a way that supports robust SCA resistance.

3.1.4 Discussion on Assumptions and Generality

It is important to emphasize that the results presented in this section are derived under a deliberately simplified set of assumptions – namely, that each logic gate has a constant and unique propagation delay, and that all primary inputs are initialized to zero. These assumptions are not intended to reflect worst-case or adversarial conditions. Rather, they serve to isolate the core structural behavior of the masking schemes under evaluation and to make the resulting glitch patterns analytically tractable. Importantly, the conclusions drawn from this simplified setting are not artifacts of the model – they expose inherent weaknesses in the logic-level design of ISW and E-ISW, independent of technology, implementation, or adversary capability.

Table 2: Glitch patterns of ISW and E-ISW AND gadgets, assuming r arrives earlier than all other primary inputs.

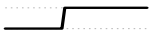
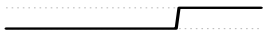
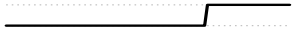
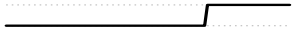
y	y^0	y^1	x^0	r	$x^0\,y^0$	$x^0\,y^1$	$x^0\,y^1 \oplus r$	ISW z^0	delayed $(x^0\,y^1 \oplus r)$	E-ISW z^0
0	0	0	0	0						
				1						
			1	0						
				1						
	1	1	0	0						
				1						
			1	0						
				1						
1	0	1	0	0						
				1						
			1	0						
				1						
	1	0	0	0						
				1						
			1	0						
				1						

Table 3: Glitch patterns of DOM AND gadget.

y	y^0	y^1	x^0	r	$x^0\,y^0$	$x^0\,y^1$	$x^0\,y^1 \oplus r$	`stored` $(x^0\,y^1 \oplus r)$	DOM z^0
0	0	0	0	0					
				1					
			1	0					
				1					
	1	1	0	0					
				1					
			1	0					
				1					
1	0	1	0	0					
				1					
			1	0					
				1					
	1	0	0	0					
				1					
			1	0					
				1					

- **Constant and unique gate delays.** Assuming that every logic gate (AND or XOR) incurs a fixed, distinct delay allows us to model the propagation of glitches in a deterministic and analyzable way. This abstraction is standard in circuit timing analysis and helps us to trace the sequence of events resulting from a given input transition. In practical implementations, however, gate delays vary due to a range of factors – input values, layout constraints, parasitic capacitances, interconnect load, manufacturing variation, temperature, voltage fluctuations, etc. These variations increase the number of possible of signal arrival times at different gates and therefore expand the space of possible glitch patterns. From a security perspective, this means that the output behavior becomes even more sensitive to subtle differences in input data and switching sequences compared to those abstracted and simplified in Table 2 and Table 1. Thus, relaxing the constant-delay assumption would not eliminate the data dependency we observe, but would instead introduce more avenues for leakage, making the actual situation worse.

- **Zero-initialization of primary inputs.** Setting all inputs to zero at the beginning of the analysis ensures that the glitch behavior is triggered by a single, well-defined transition, allowing for clearer interpretation of the circuit's response. This is a common strategy in hardware verification and SCA modeling, where one aims to isolate the contribution of specific transitions to observed power or EM traces. A more exhaustive analysis would involve transitioning between all possible input pairs and exploring how every such change propagates through the circuit. While such an analysis would be more comprehensive, it would also significantly complicate the characterization without altering the fundamental takeaway: even in this minimal case, data-dependent glitch propagation is evident.

From another angle, these simplifying assumptions can be seen as defining a *clean room* environment in which structural flaws are easier to detect. If a scheme fails under idealized timing conditions and single-transition inputs, then it also fails under real operating conditions where multiple transitions, delay variations, crosstalk, and capacitive effects are all present. Moreover, while our analysis focuses on leakage through glitches and their dependency on sensitive variables, real-world power and EM measurements will inevitably integrate such effects over time – further amplifying the statistical bias introduced by data-dependent glitches.

In summary, the modeling assumptions used in this section do not undermine the generality of the findings – they make the root causes of insecurity more transparent. By removing confounding variables, we expose the logical pathways through which leakage occurs and demonstrate that the E-ISW construction, despite its intended improvements over ISW, fails to block these paths effectively. In more realistic or adversarial settings, these vulnerabilities are not only expected to persist but may intensify.

3.2 Experimental

In addition to the theoretical analysis presented earlier, we conducted an extensive set of experiments to empirically compare the security of the ISW, E-ISW, and DOM masking schemes in practice. These experiments aim to evaluate the presence of first-order leakage in concrete hardware realizations using well-established Side-Channel Analysis (SCA) evaluation techniques. While our theoretical model is designed to isolate structural flaws, practical measurements capture real-world effects such as routing-induced skew, parasitic capacitance, glitches, and measurement noise – factors that are often difficult to model precisely but highly relevant to leakage.

3.2.1 Experimental Setup

Our experiments were carried out on a SAKURA-G evaluation platform [SAK16], which features a Xilinx Spartan-6 FPGA and is widely used in the side-channel research community due to its low-noise, measurement-friendly design. We synthesized and deployed standalone implementations of the ISW, E-ISW, and DOM masked AND gates on the board, ensuring comparable placement and routing constraints across designs to reduce systematic bias.

To monitor power consumption, we connected a digital oscilloscope to the $1\,\Omega$ shunt resistor mounted in the Vdd path of the FPGA. To improve the signal-to-noise ratio – especially important for measuring small, localized computations – we leveraged the SAKURA-G's on-board AC amplifier. This amplifier filters out the low-frequency noise (including the DC shift) and enhances signal variations caused by dynamic power consumption of the circuit under evaluation.

Measurements were acquired at a sampling rate of 500 MS/s while clocking the FPGA at 6 MHz. This relatively low clock frequency is a common choice in SCA evaluations, as it stretches the clock cycles and allows better isolation of the power consumption corresponding to each evaluation step. In particular, it helps prevent overlapping of power peaks of adjacent clock cycles, which could otherwise blur leakage effects and lead to false negatives during statistical testing.

For leakage detection, we applied the widely accepted fixed-versus-random Test Vector Leakage Assessment (TVLA) methodology [SM15, CDG+13]. In each measurement cycle, the device was supplied with either a fixed input vector or a randomly generated one, chosen with equal probability. All input shares were Boolean-masked using fresh random values for each trace to conduct the analysis under the intended masking behavior.

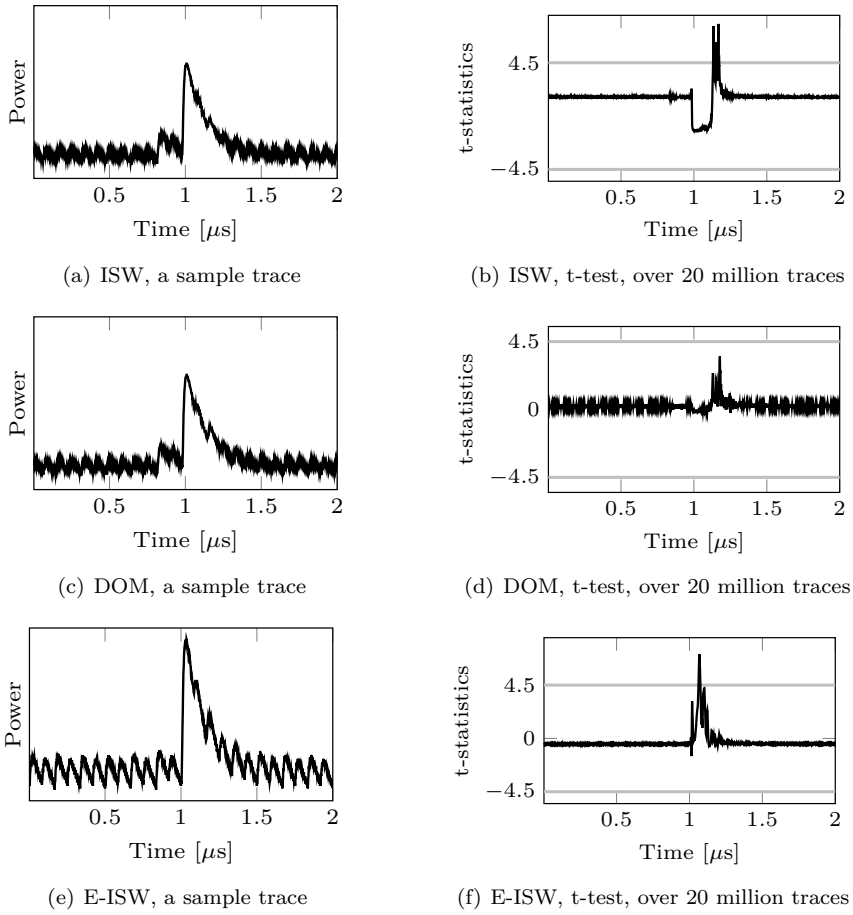
Under the assumption of first-order secure masking, the mean power consumption should be statistically indistinguishable between the fixed and random input classes. To test this, we applied Welch's t-test over aligned time samples of the power traces, which is known to examine the existence of first-order leakages. Any statistically significant deviation of t-statistics from 4.5 indicates that information about the underlying unmasked variable may be leaked through first-order power consumption.

3.2.2 Implementation Details and Results

To enable a fair and controlled comparison across masking schemes, we implemented two distinct designs for each of the ISW, E-ISW, and DOM gadgets, reflecting both parallel and compositional usage scenarios commonly found in cryptographic hardware.

- **Parallel Gadget Array.**
  In the first design, we instantiated 128 masked AND gadgets operating in parallel. Each gadget computed one bit of a 128-bit vector AND operation, effectively implementing $z_i = x_i\, y_i$ for $i = 0, \ldots, 127$, with all operations masked and evaluated simultaneously. Each gadget required a fresh random mask in every clock cycle, resulting in 128 random bits per cycle. To generate this entropy in a scalable manner, we integrated an unrolled Trivium instance as a Pseudo-Random Number Generator (PRNG), following the high-throughput design proposed in [CMM+24]. The PRNG was clocked in sync with the gadgets' logic and produced 128 fresh random bits per clock cycle.

  For the E-ISW gadgets, we faithfully followed the timing assumptions of [TBE+24], in particular enforcing delayed arrival of the randomized terms $x^0\, y^1 \oplus r$ and $x^1\, y^0 \oplus r$ relative to the non-blinded partial products $x^0\, y^0$ and $x^1\, y^1$. This was achieved by inserting ten inverter stages per delay line (see Figure 2). To ensure that synthesis tools do not eliminate or compress these delay structures, we preserved the module hierarchy and applied the `KEEP` and `DONT_TOUCH` attributes to every inverter instance.



(a) ISW, a sample trace

(b) ISW, t-test, over 20 million traces

(c) DOM, a sample trace

(d) DOM, t-test, over 20 million traces

(e) E-ISW, a sample trace

(f) E-ISW, t-test, over 20 million traces

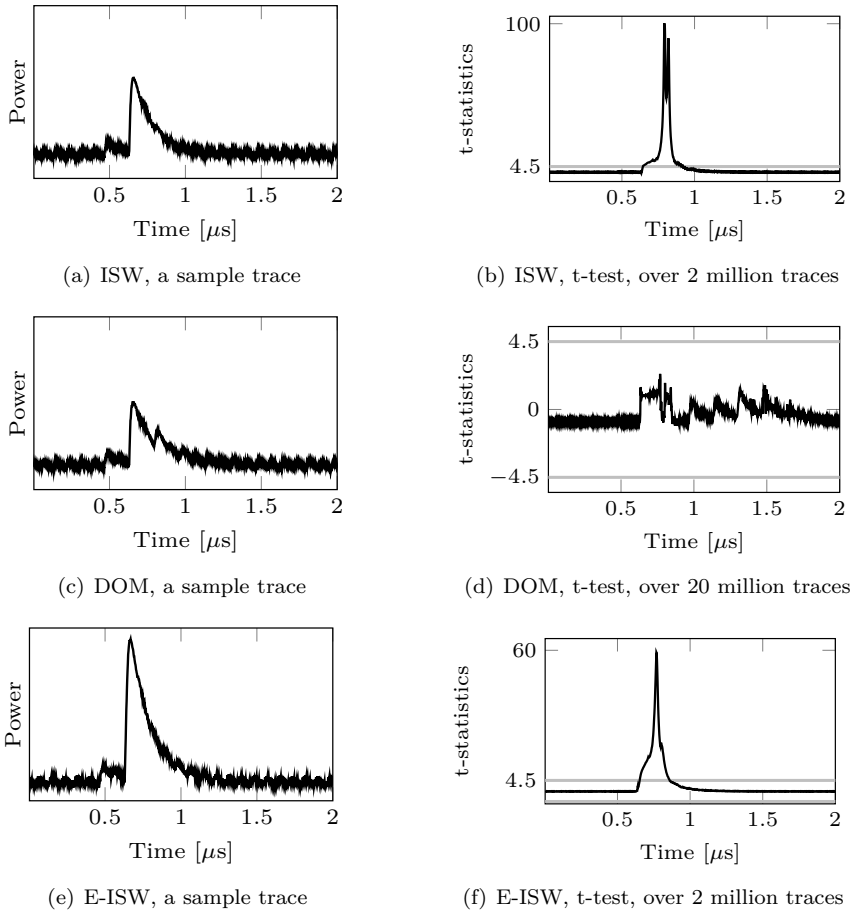
Figure 4: Experimental analysis results for the 128 parallel AND gadgets.

This guaranteed that each inverter was mapped to a distinct physical Look-Up Table (LUT), maintaining consistent delay behavior in hardware.

Figure 4 summarizes the results of the TVLA analysis based on 20 million collected power traces per design. As expected, the DOM implementation exhibits no statistically significant leakage, with all t-statistics remaining below the standard threshold of $|t| = 4.5$. In contrast, both ISW and E-ISW failed the test, demonstrating clear and reproducible first-order leakage. This result confirms that the theoretical vulnerabilities identified in earlier sections manifest under practical conditions, even when the delay assumptions of E-ISW are carefully enforced.

- **Cascaded Composition.**
  In the second set of designs, we evaluated each scheme in a compositional setting to reflect a more realistic cryptographic usage scenario, such as the construction of S-boxes or logic trees. Specifically, we implemented a reduction tree of AND operations: a 128-bit input vector was reduced to a single output bit through seven layers of masked AND gadgets. The first layer performs 64 ANDs on input pairs, followed by 32 ANDs in the second layer, and so on, culminating in a single output at the root. In total, this structure requires 127 masked gadgets and, consequently, 127 fresh random masks per clock cycle.

  The TVLA results for this cascaded configuration are shown in Figure 5. Consistent



(a) ISW, a sample trace

(b) ISW, t-test, over 2 million traces

(c) DOM, a sample trace

(d) DOM, t-test, over 20 million traces

(e) E-ISW, a sample trace

(f) E-ISW, t-test, over 2 million traces

Figure 5: Experimental analysis results for the 128-to-1 tree of AND gadgets.

with our expectations, the DOM implementation remained secure and passed the t-test throughout. Both ISW and E-ISW, however, exhibited very strong leakage even with significantly fewer traces compared to the former case study, i.e. the 128 parallel gadget array. In fact, the leakage was so prominent that we terminated measurements for ISW and E-ISW after collecting only 2 million traces, as their t-statistics had already far exceeded the detection threshold. This highlights that cascading unregistered gadgets – especially in deeper logic trees – amplifies timing mismatches and glitch propagation, exacerbating the security deficiencies of such flawed masking schemes.

3.2.3 Observations and Comparison

Beyond the t-test results, we make two additional observations regarding the physical behavior and relative leakage characteristics of the evaluated designs.

- **Power consumption and circuit activity.** For visual consistency and interpretability, all trace figures (Figure 4 through Figure 5) share a common y-axis scale. This allows a direct comparison of the dynamic power consumption across masking schemes. It is apparent that both ISW and E-ISW consume substantially more power than DOM, particularly in the cascaded configuration. This difference is attributed

to the absence of internal registers in ISW and E-ISW, which allows uncontrolled glitch propagation throughout the logic. In contrast, DOM employs clocked registers that suppress spurious transitions and synchronize signal changes, thereby reducing the number of active transitions per clock cycle. Additionally, the inverter chains used to enforce delays in E-ISW significantly increase switching activity and dynamic power, making it the most power-hungry among the three.

- **Relative leakage of ISW vs. E-ISW.** Although both ISW and E-ISW clearly fail the first-order TVLA tests, we note that the t-statistics for E-ISW is consistently lower than those of ISW. This suggests that the delay synchronization introduced in E-ISW has a marginal positive effect in reducing leakage amplitude – possibly by aligning certain transitions more closely in time and suppressing specific glitch interactions. However, the observed improvement is purely quantitative rather than qualitative. E-ISW still leaks significantly, with t-statistics exceeding the standard detection threshold by a wide margin. Therefore, it cannot be considered secure in practice, nor does it offer a meaningful advantage over the original ISW scheme. These findings are fully consistent with our theoretical results: although E-ISW attempts to address glitch-induced leakage by controlling signal arrival, it fails to eliminate the underlying cause – namely, unregistered intermediate computation paths that remain sensitive to data-dependent delays.

- **Generality across security orders.** Our analyses focused exclusively on first-order versions of gadgets (E-ISW, etc.) to enable clearer interpretation and more efficient testing. However, the structural issues we identify – namely, data-dependent glitch propagation due to unregistered paths and uncontrolled delays – persist at higher security orders as well. Indeed, we have observed similar leakage behaviors in higher-order instantiations, though we omit detailed results for brevity. In short, the findings reported for the first-order case are representative of a broader class of designs, and our conclusions remain valid in more general settings.

- **Security under formal leakage models.** Recent work on robust but relaxed probing models [MM24] offers a refined alternative to the standard robust probing model by explicitly incorporating temporal and glitch- and transition-based effects into the adversarial view. When applied to E-ISW, such models would correctly identify its vulnerability, since the masked computation paths remain data-dependent and unregistered. To the best of our knowledge, no known formal leakage model – whether conservative or relaxed – would classify E-ISW as secure.

4 Conclusions

The E-ISW scheme was recently introduced as a hardware-level enhancement of the classic ISW masking scheme, aiming to avoid glitch-induced leakage by enforcing specific signal arrival patterns through controlled delays. In this work, we have presented a comprehensive and critical evaluation of E-ISW, combining structural analysis with empirical measurements on FPGA hardware.

Our theoretical investigation, conducted under a carefully designed yet conservative model, revealed that E-ISW fails to eliminate data-dependent glitch patterns in its output signals. Despite incorporating additional delay logic to coordinate the arrival of intermediate signals, the construction remains vulnerable to first-order leakage arising from unregistered data paths. This leakage persists under both early and late arrival scenarios of the refresh input, and is fundamentally rooted in the structural design of the masking scheme.

To validate these findings, we performed extensive experimental tests using the standard fixed-versus-random TVLA methodology on real FPGA implementations of ISW, E-ISW,

and DOM. Our results confirm that both ISW and E-ISW consistently exhibit first-order leakage, when evaluated in both parallel and cascaded configurations. In contrast, the DOM scheme – with its systematic use of synchronous registers – exhibited no detectable leakage, underscoring the effectiveness of well-structured design principles in hardware masking.

Taken together, our results decisively show that E-ISW does not provide a secure fix to ISW, and falls short of its intended goal of practical glitch resistance. While its use of delayed signal paths may reduce certain timing skews, it does not address the root cause of leakage – namely, data-dependent transitions in unregistered logic. This re-highlights a broader lesson for the hardware security community: heuristic countermeasures that are not grounded in formal compositional reasoning can yield misleading security guarantees. Leakage detection methods alone – especially when limited in scope – are insufficient to validate masking schemes in the presence of complex hardware effects such as glitch propagation and logic composition.

Going forward – as learned during last decade – secure hardware masking must rely on rigorously defined design patterns, composability-aware constructions, and formal verification techniques that reflect realistic hardware behavior. Only then can we ensure that masking-based countermeasures provide the robust protection needed against physical adversaries in practice.

Acknowledgments

The work described in this paper has been supported in part by the German Research Foundation (DFG) through the project 549340884 “Maturing Physical Security Models in Realistic Scenarios”.

References

[BCO04] Eric Brier, Christophe Clavier, and Francis Olivier. Correlation Power Analysis with a Leakage Model. In *CHES 2004*, volume 3156 of *Lecture Notes in Computer Science*, pages 16–29. Springer, 2004.

[BDF+17] Gilles Barthe, François Dupressoir, Sebastian Faust, Benjamin Grégoire, François-Xavier Standaert, and Pierre-Yves Strub. Parallel Implementations of Masking Schemes and the Bounded Moment Leakage Model. In *EUROCRYPT 2017*, volume 10210 of *Lecture Notes in Computer Science*, pages 535–566, 2017.

[BWG+22] Arthur Beckers, Lennert Wouters, Benedikt Gierlichs, Bart Preneel, and Ingrid Verbauwhede. Provable Secure Software Masking in the Real-World. In *COSADE 2022*, volume 13211 of *Lecture Notes in Computer Science*, pages 215–235. Springer, 2022.

[CDG+13] Jeremy Cooper, Elke DeMulder, Gilbert Goodwill, Joshua Jaffe, Gary Kenworthy, Pankaj Rohatgi, et al. Test vector leakage assessment (TVLA) methodology in practice. In *International Cryptographic Module Conference*, volume 20, 2013.

[CGLS21] Gaëtan Cassiers, Benjamin Grégoire, Itamar Levi, and François-Xavier Standaert. Hardware Private Circuits: From Trivial Composition to Full Verification. *IEEE Trans. Computers*, 70(10):1677–1690, 2021.

[CJRR99] Suresh Chari, Charanjit S. Jutla, Josyula R. Rao, and Pankaj Rohatgi. Towards sound approaches to counteract power-analysis attacks. In *CRYPTO '99*, volume 1666 of *Lecture Notes in Computer Science*, pages 398–412. Springer, 1999.

[CMM+24] Gaëtan Cassiers, Loïc Masure, Charles Momin, Thorben Moos, Amir Moradi, and François-Xavier Standaert. Randomness Generation for Secure Hardware Masking - Unrolled Trivium to the Rescue. *IACR Commun. Cryptol.*, 1(2):4, 2024.

[CSV24] Gaëtan Cassiers, François-Xavier Standaert, and Corentin Verhamme. Low-Latency Masked Gadgets Robust against Physical Defaults with Application to Ascon. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(3):603–633, 2024.

[FGP+18] Sebastian Faust, Vincent Grosso, Santos Merino Del Pozo, Clara Paglialonga, and François-Xavier Standaert. Composable Masking Schemes in the Presence of Physical Defaults & the Robust Probing Model. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(3):89–120, 2018.

[GD23] John Gaspoz and Siemen Dhooghe. Threshold Implementations in Software: Micro-architectural Leakages in Algorithms. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(2):155–179, 2023.

[GIB18] Hannes Groß, Rinat Iusupov, and Roderick Bloem. Generic Low-Latency Masking in Hardware. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(2):1–21, 2018.

[GMK16] Hannes Groß, Stefan Mangard, and Thomas Korak. Domain-Oriented Masking: Compact Masked Hardware Implementations with Arbitrary Protection Order. In *TIS@CCS 2016*, page 3. ACM, 2016.

[HHB24] Johannes Haring, Vedad Hadzic, and Roderick Bloem. Closing the Gap: Leakage Contracts for Processors with Transitions and Glitches. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(4):110–132, 2024.

[HRG14] Annelie Heuser, Olivier Rioul, and Sylvain Guilley. Good Is Not Good Enough - Deriving Optimal Distinguishers from Communication Theory. In *CHES 2014*, volume 8731 of *Lecture Notes in Computer Science*, pages 55–74. Springer, 2014.

[ISW03] Yuval Ishai, Amit Sahai, and David A. Wagner. Private Circuits: Securing Hardware against Probing Attacks. In *CRYPTO 2003*, volume 2729 of *Lecture Notes in Computer Science*, pages 463–481. Springer, 2003.

[KJJ99] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In *CRYPTO '99*, volume 1666 of *Lecture Notes in Computer Science*, pages 388–397. Springer, 1999.

[KM22] David Knichel and Amir Moradi. Low-Latency Hardware Private Circuits. In *CCS 2022*, pages 1799–1812. ACM, 2022.

[KSM22] David Knichel, Pascal Sasdrich, and Amir Moradi. Generic Hardware Private Circuits Towards Automated Generation of Composable Secure Gadgets. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(1):323–344, 2022.

[MM24] Nicolai Müller and Amir Moradi. Robust but Relaxed Probing Model. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(4):451–482, 2024.

[MOP07] Stefan Mangard, Elisabeth Oswald, and Thomas Popp. *Power analysis attacks - revealing the secrets of smart cards*. Springer, 2007.

[MPO05] Stefan Mangard, Norbert Pramstaller, and Elisabeth Oswald. Successfully Attacking Masked AES Hardware Implementations. In *CHES 2005*, volume 3659 of *Lecture Notes in Computer Science*, pages 157–171. Springer, 2005.

[MPW22] Ben Marshall, Dan Page, and James Webb. MIRACLE: MIcRo-ArChitectural Leakage Evaluation A study of micro-architectural power leakage across many devices. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(1):175–220, 2022.

[NGPM22] Rishub Nagpal, Barbara Gigerl, Robert Primas, and Stefan Mangard. Riding the Waves Towards Generic Single-Cycle Masking in Hardware. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(4):693–717, 2022.

[NRS11] Svetla Nikova, Vincent Rijmen, and Martin Schläffer. Secure Hardware Implementation of Nonlinear Functions in the Presence of Glitches. *J. Cryptol.*, 24(2):292–321, 2011.

[PM09] Emmanuel Prouff and Robert P. McEvoy. First-Order Side-Channel Attacks on the Permutation Tables Countermeasure. In *CHES 2009*, volume 5747 of *Lecture Notes in Computer Science*, pages 81–96. Springer, 2009.

[RBN+15] Oscar Reparaz, Begül Bilgin, Svetla Nikova, Benedikt Gierlichs, and Ingrid Verbauwhede. Consolidating Masking Schemes. In *CRYPTO 2015*, volume 9215 of *Lecture Notes in Computer Science*, pages 764–783. Springer, 2015.

[SAK16] SAKURA. Side-channel Attack User Reference Architecture. http://satoh.cs.uec.ac.jp/SAKURA/index.html, 2016.

[SBHM20] Pascal Sasdrich, Begül Bilgin, Michael Hutter, and Mark E. Marson. Low-Latency Hardware Masking with Application to AES. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2020(2):300–326, 2020.

[SM15] Tobias Schneider and Amir Moradi. Leakage Assessment Methodology - A Clear Roadmap for Side-Channel Evaluations. In *CHES 2015*, volume 9293 of *Lecture Notes in Computer Science*, pages 495–513. Springer, 2015.

[SM21] Aein Rezaei Shahmirzadi and Amir Moradi. Re-Consolidating First-Order Masking Schemes Nullifying Fresh Randomness. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(1):305–342, 2021.

[TBE+24] Sofiane Takarabt, Javad Bahrami, Mohammad Ebrahimabadi, Sylvain Guilley, and Naghmeh Karimi. Securing ISW Masking Scheme Against Glitches. In *DATE 2024*, pages 1–2. IEEE, 2024.

[VDB+24] Dilip Kumar S. V., Siemen Dhooghe, Josep Balasch, Benedikt Gierlichs, and Ingrid Verbauwhede. Time Sharing - A Novel Approach to Low-Latency Masking. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(3):249–272, 2024.

[ZM24] Jannik Zeitschner and Amir Moradi. PoMMES: Prevention of Micro-architectural Leakages in Masked Embedded Software. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(3):342–376, 2024.

[ZSS+21] Sara Zarei, Aein Rezaei Shahmirzadi, Hadi Soleimany, Raziyeh Salarifard, and Amir Moradi. Low-Latency Keccak at any Arbitrary Order. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(4):388–411, 2021.